

Technical Evaluation Report: Autonomous CI/CD Build Assistant

Saim Jawed
saim.jawed@accenture.com
Accenture AEH Internship

June 29, 2026

Abstract

This report evaluates the accuracy, precision, and helpfulness of the Autonomous CI/CD Build Assistant across a dataset of 20 distinct, realistic build failures, processed directly by the LLMClient.

1 Methodology

I collected 20 failing CI/CD pipeline logs representing common edge cases in a standard software environment (including Python Syntax errors, Node dependency conflicts, OOM limits, and Docker repository timeouts). Each log was fed to the Google Gemini agent, and the resulting FailureDiagnosis JSON was evaluated on three criteria:

1. **Categorization Accuracy:** Did the AI select the correct `failure_type`?
2. **False Positive Rate:** Did the AI hallucinate a code fix for an infrastructure problem?
3. **Actionability (Helpfulness):** Were the `file_changes` correctly mapped and populated?

2 Overall Results

Overall Success Rate: The AI successfully analyzed all 20 logs and successfully distinguished between code bugs (which require PR patches) and transient network errors (which require autonomous pipeline retries).

3 Real Examples of Good Outputs (From Execution Data)

3.1 Example 1: Authentic Infrastructure Recognition (No False Positives)

Scenario (Log ID 13):
`requests.exceptions.ConnectionError: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded`
Real AI Diagnosis:

```
{
  "failure_types": ["network_timeout"],
  "root_cause": "The application is attempting to connect to a service at localhost:8080, but the service is not running or is unreachable.",
  "confidence": "HIGH",
  "file_changes": 0
}
```

Evaluation: Perfect. The AI correctly identified a `network.timeout` and returned **0 file changes**. This is critical safety behavior, it prevents the agent from hallucinating code changes for transient CI infrastructure issues, safely triggering `retry.yml` instead.

3.2 Example 2: Authentic Code Bug Detection

Scenario (Log ID 1): `Python SyntaxError: expected ':' in a function declaration.`
Real AI Diagnosis:

```
{
  "failure_types": ["compile_error"],
  "root_cause": "Missing colon at the end of the function definition line in app.py",
  "confidence": "HIGH",
  "file_changes": 1
}
```

Evaluation: Perfect. The AI successfully categorized the missing syntax as a `compile.error` and

Metric	Score	Notes
Categorization Accuracy	95% (19/20)	High accuracy on syntax, network timeouts, and dependency errors. One generic runtime error was classed as a <code>test_failure</code> .
False Positive Rate	0% (0/20)	The agent correctly refused to suggest code fixes for transient infrastructure network timeouts.
Patch Actionability	100% (18/18)	For the 18 logs that represented actual code/config issues, the agent successfully produced <code>file_changes</code> patches.

Table 1: Aggregate evaluation metrics across 20 failure logs.

correctly populated the `file_changes` payload to be picked up by the `/apply-fix` command.

3.3 Example 3: Context Limitation (The “Deep Scan” Requirement)

Scenario (Log ID 17): Python `MemoryError` when allocating a massive list.

Real AI Diagnosis:

```
{
  "failure_types": ["oom_error"],
  "root_cause": "The application is
    attempting to allocate a list of 1
    billion integers, which exceeds the
    available system memory.",
  "confidence": "HIGH",
  "file_changes": 1
}
```

Evaluation: Good, but requires deep scanning. The AI correctly categorized this as an `oom_error`. However, while it can point out the offending `10**9` list comprehension, solving complex memory leaks often requires full file context. This validates my design decision to include the `/deep-scan` manual override command for developers.

4 Conclusion

The AI Build Assistant excels at catching deterministic bugs (syntax, configuration) and accurately separating them from transient network and memory failures (`network.timeout`, `oom_error`). By correctly refusing to patch infrastructure errors, it achieves a **0% false-positive code hallucination rate**, making it a highly safe agent to deploy in production CI/CD environments.